# Project Walkthrough & Interview Preparation Guide

## US Accidents Statistical Analysis — Built Step by Step for a Beginner

This document explains, in plain language, everything that was done to build this project — not just *what* was built, but *why* each decision was made, in the order it happened. It ends with a set of interview questions (with answers) that a hiring manager might reasonably ask about a project like this one.

---

## Table of contents



---

## 1. What this project is

A statistical analysis of ~7.7 million real US traffic accident records (2016–2023), built as a portfolio project to demonstrate Data Analyst/BI skills to a prospective employer. The final deliverables are:

- A **Jupyter notebook** with the full statistical analysis
- A **live, interactive web dashboard**
- Two **written reports** (a short one and a deep-dive one)
- A **public GitHub repository** containing everything, with automated tests and automatic deployment

The goal wasn't just to produce charts — it was to show real statistical reasoning (hypothesis tests, not just "eyeballing" a chart) and real software engineering practice (tests, version control, automation).

---

## Step 1: Choosing the dataset

**What we did:** Picked the "US Accidents (2016–2023)" dataset from Kaggle — about 7.7 million rows, 1–3GB.

**Why it matters:** A portfolio project's dataset choice matters a lot. We specifically avoided small, overused "toy" datasets (like Titanic or Iris) that every beginner uses, because they don't demonstrate the ability to handle real-world data volume. We also avoided picking something *too* large and unwieldy for the time available (a few days). US Accidents was a good middle ground: large and genuinely impressive by scale, but with a clear, relatable business story (road safety) that a Data Analyst/BI interviewer would immediately understand and care about.

**Key beginner concept — matching the dataset to the audience:** If you're applying for a Data Analyst/BI role, your project's dataset should tell a *business* story (what should we do differently because of this data?), not just be a machine learning exercise.

---

## Step 2: Setting up version control (Git and GitHub)

**What we did:** 1. Created a brand-new GitHub account (`dsamy-byte`), kept completely separate from any personal GitHub account. 2. Generated a dedicated SSH key just for this project, instead of reusing an existing one. 3. Added an entry to the local SSH config file that tells the computer "when talking to this specific repo, use this specific key" — so there's no need to manually switch accounts every time we push code. 4. Set the Git username/email *only* for this one project folder (`git config` without `--global`), so it doesn't affect any other projects on the same computer. 5. Initialized the repository (`git init`) and made the first commit.

**Why it matters (beginner explanation):** - **Git** is a tool that tracks every change to your code over time, so you can always go back, compare versions, and collaborate safely. - **GitHub** is a website that hosts your Git repository online, so other people (like an employer) can view your code. - **SSH keys** are how your computer proves its identity to GitHub without typing a password every time. A "host alias" lets one computer cleanly manage multiple separate GitHub identities (e.g., a personal account and a project-specific account) without them interfering with each other. - Keeping a "just for this project" identity separate from a personal one is good practice when you want a clean, professional, dedicated presence for something you're going to show an employer.

---

## Step 3: Setting up the Python environment

**What we did:** Used a tool called `uv` (a fast, modern Python package and environment manager) to set up the project. This included writing a `pyproject.toml` file listing every library the project needs (DuckDB, pandas, scipy, statsmodels, Streamlit, etc.) with minimum version numbers, then running `uv sync` to install everything into an isolated environment (a `.venv` folder) just for this project.

**A real problem we hit and solved:** Installing all these packages was unexpectedly slow. Investigating showed the project lived on an external hard drive formatted as **exFAT** — a filesystem type that (unlike the more common NTFS) doesn't support a speed trick called "hardlinking." Normally, `uv` can install a package almost instantly by creating a lightweight link to an already-downloaded copy instead of copying the whole file. exFAT doesn't support this at all, so every install had to do a full, slower file copy instead. We diagnosed this by reading `uv`'s own warning

message and checking the drive's filesystem type directly, then made a deliberate decision to accept the slower installs rather than restructure the whole project just to work around it.

**Why it matters (beginner explanation):** This is a great example of real-world debugging: don't just try random fixes — read the actual error/warning message, form a hypothesis (e.g., "maybe it's about which drive the cache is on"), test it, and if that's wrong, dig one level deeper (in this case, the *type* of filesystem, not just which drive) until you find the real cause. Then weigh whether fixing it is worth the effort versus just accepting a known, understood tradeoff.

---

## Step 4: Building the data pipeline with DuckDB

**What we did:** Wrote a Python module (`ingest.py`) that uses **DuckDB** to read the accident CSV file and check that it has all the columns the analysis needs, before doing anything else with it.

**Why DuckDB instead of just pandas?** pandas is the most common Python tool for working with tabular data, but it generally needs to load the *entire* file into computer memory (RAM) before you can do anything with it. A 3GB, 7.7-million-row file might not fit comfortably in memory, and even if it does, it can be slow. DuckDB is a different kind of tool — an "embedded analytical database" — that can query a huge CSV file directly off the disk, similar to how a full database server works, but without needing to install or run a separate server. It's extremely fast at the kind of "group by and count" operations this project needed.

**Why validate the schema first?** If the CSV file were missing an expected column (e.g., if Kaggle changed the file format), we wanted the program to fail immediately with a clear error message ("Missing required columns: [...]") rather than crash confusingly deep inside a later calculation.

---

## Step 5: Aggregation and sampling strategy

**What we did:** Instead of working with all 7.7 million rows for every single operation, we built two kinds of smaller, derived datasets:

1. **Aggregated rollups** — e.g., "how many accidents happened in each state, and what was the average severity?" This produces a table with only ~50 rows (one per state) instead of millions.
2. **A random sample** — 200,000 rows, randomly selected but in a *reproducible* way (using a fixed "seed" number, 42, so re-running the selection always picks the same 200,000 rows).

**Why sample instead of using everything?** For statistical hypothesis testing, you generally don't need every single row to get a reliable, valid answer — a large, well-chosen random sample gives you almost the same statistical power with far less computing cost. 200,000 rows is still very large by normal statistics standards (most textbook examples use hundreds of rows, not hundreds of thousands).

**Why does the sample need to be reproducible?** If you ran the same analysis twice and got two different samples, you might get two slightly different results, which would be confusing and undermine trust in the findings. Using a fixed seed means anyone who runs this project's code gets the *exact same* sample, and therefore the exact same results, every time.

---

## Step 6: The statistics module

We wrote three reusable statistical functions, each wrapping a well-established method:

### Chi-square test

**What it answers:** "Are two categorical variables related to each other, or are they independent?" We used it to test whether accident severity (a category: 1, 2, 3, or 4) is related to weather condition (also a category: Clear, Rain, Fog, etc.).

### ANOVA (Analysis of Variance)

**What it answers:** "Does the average of a numeric variable differ across several groups?" We used it to test whether the average accident severity differs depending on the hour of day.

### Logistic regression

**What it answers:** "Given several input factors, how much does each one change the odds of a particular outcome happening?" We used it to predict whether an accident is "high severity" (severity 3 or 4) based on whether it happened near a junction, crossing, traffic signal, or stop sign — and to report each factor's effect as an easy-to-interpret **odds ratio** (see the interview Q&A section for a plain explanation of what an odds ratio means).

**Why wrap these in our own functions instead of calling the statistics library directly every time?** So the rest of the project (the notebook, future analyses) can just call `chi_square_severity_by_weather(data)` and get back a clean, consistent result — the messy statistical library details are handled in one place, tested once, and reused everywhere.

---

## Step 7: Test-driven development — and the real bugs it caught

**What we did:** For every piece of code, we wrote the *test* first (describing exactly what the correct behavior should look like), watched it fail (since the code didn't exist yet), then wrote the minimum code needed to make the test pass. This is called **Test-Driven Development (TDD)**.

**This wasn't just theoretical — it caught two real bugs immediately:**

1. **A DuckDB bug.** Our first version of the data-loading code tried to safely insert the file path into a SQL command using a "prepared parameter" (a standard, safe way to avoid SQL injection bugs). But it turned out DuckDB doesn't allow prepared parameters inside a `CREATE VIEW` statement — only in simpler `SELECT` statements. The test failed immediately with a clear database error, which we would have discovered eventually anyway, but TDD caught it in seconds, before it was buried under more code.
2. **A subtle Python/numpy bug.** Our statistics functions computed "is this result significant?" as `p_value < 0.05`. This looks like it produces a normal `True`/`False`, but because `p_value` came from a scientific computing library (`scipy`), the result was actually a special `numpy.bool_` type, not Python's built-in `bool`. A test that checked `result["significant"] is True` (checking *identity*, not just equality) failed, revealing the subtle type mismatch — something that could have caused confusing bugs much later (e.g., if that value were ever compared with `is` somewhere else, or serialized to JSON).

**Why this matters (beginner explanation):** Tests aren't just about "proving your code works" — writing them *first* forces you to think precisely about what "correct" means before you start coding, and they catch real, subtle mistakes immediately, when they're cheap to fix, instead of much later when they're expensive and confusing to

track down.

---

## Step 8: Documenting the project as we went

**What we did:** Kept a running set of documentation files (an "Obsidian vault" — a folder of linked Markdown notes) alongside the code:

- **Data-Dictionary.md** — explains what every column in the dataset means.
- **Methodology-Log.md** — explains exactly how the data was acquired and processed.
- **Decisions.md** — records *why* every major choice was made (e.g., why DuckDB and not a database server, why Render and not Docker).
- **Findings.md** — the actual statistical results, updated as soon as they were computed.

**Why document decisions, not just results?** Anyone reviewing this project later (including an interviewer) can see not just *what* was built, but the *reasoning* behind it — which is usually what distinguishes a junior analyst from a more experienced one.

---

## Step 9: Running the pipeline on the real 7.7-million-row dataset

Once the pipeline was built and tested (using a small, fake 12-row sample file for fast, safe testing), we ran it for real against the actual downloaded 3GB CSV file. This took under a minute — DuckDB efficiently scanned the whole file and produced:

- Rollup tables by state, weather condition, hour of day, and year
- The 200,000-row random sample
- A small JSON file summarizing the logistic regression results (used later by the dashboard)

These outputs are small (a few megabytes total) and were committed to GitHub — but the original 3GB raw file was **not** committed (it's far too large, and GitHub itself blocks files over 100MB).

---

## Step 10: The analysis notebook, explained

The Jupyter notebook (`01_eda_and_hypothesis_testing.ipynb`) tells the full statistical story, step by step:

1. **Load the small pre-built rollups and sample** (never the raw file).
2. **Descriptive charts** — which states have the most accidents, how severity trends across the day.
3. **Hypothesis test 1** — is severity related to weather? (Yes, strongly — see interview Q&A for what this means.)
4. **Hypothesis test 2** — does severity vary by hour of day? (Yes.)
5. **Logistic regression** — which road features predict high-severity accidents? All four tested features (junction, crossing, traffic signal, stop sign) turned out to be statistically significant predictors.

A deliberate design choice: every interpretive sentence in the notebook (e.g., "Severity IS significantly associated with weather condition...") is generated *by code*, using the real computed numbers, rather than typed by hand. This

guarantees the written narrative can never drift out of sync with the actual results, even if the analysis is re-run later with updated data.

---

## Step 11: Building the interactive dashboard

**What we did:** Built a **Streamlit** dashboard — a Python tool that turns a data analysis script into an interactive website without needing to know HTML/CSS/JavaScript. The dashboard includes:

- A **US map**, colored by accident count per state
- A **year-over-year trend line**
- **Cross-filterable** weather and hour-of-day charts — pick a state and/or year from a dropdown, and the charts update live
- A **"key drivers" panel** showing the logistic regression odds ratios visually, with confidence intervals

**Why Streamlit and not Tableau/Power BI?** The employer specifically asked for a *Python* project, and Streamlit produces a genuinely interactive, code-driven dashboard rather than a static export.

**An important design decision:** The dashboard never touches the raw 3GB file — it only reads the small, pre-built rollup files. This keeps it fast even though it's built on top of a massive dataset.

---

## Step 12: Continuous Integration (CI)

**What we did:** Set up **GitHub Actions**, a free automation service built into GitHub. We wrote a configuration file (`.github/workflows/ci.yml`) that says: "every time code is pushed to this repository, automatically install the project and run all its automated tests and code-quality checks."

**Why this matters (beginner explanation):** Without CI, tests only run when a person remembers to run them manually. With CI, there's no way to accidentally push broken code without immediately finding out — GitHub will show a red ✗ if anything fails. This is standard practice at every professional software team.

---

## Step 13: Continuous Deployment (CD) to Render

**What we did:** Connected the GitHub repository to **Render**, a cloud hosting service, using a configuration file (`render.yaml`) that tells Render exactly how to install and start the dashboard. Render was set to "auto-deploy" — meaning every time new code is pushed to the `main` branch on GitHub, Render automatically rebuilds and republishes the live dashboard, with no manual steps required.

**A decision worth explaining:** We initially considered packaging the app with Docker (a way of bundling an app with its exact environment into a portable "container"), but decided against it — Render can run a plain Python app directly with a simple build/start command, which is simpler to maintain and unnecessary complexity for this project's needs.

**Why CI and CD together matter:** This is what people mean by "CI/CD" — a fully automated pipeline from `git push` to a live, tested, deployed application, with no manual steps and no risk of forgetting to test or deploy something.

## Step 14: Writing reports for different audiences

We produced three different written deliverables, each for a different purpose:

1. `executive_summary` — short, plain-English, for a reader who just wants the headline findings without technical detail.
2. `comprehensive_report` — the deep-dive version, including the full notebook code and output, the data dictionary, and every project decision — for a reader (or interviewer) who wants to verify the actual work, not just trust a summary.
3. **This document** — a step-by-step explanation of the *process*, aimed at helping the project's author explain and defend it in an interview.

**Why write more than one report?** Different readers want different levels of depth. A hiring manager skimming twenty portfolios wants the 30-second version; a technical interviewer who's genuinely evaluating your skills wants to see the real work.

---

## Full project timeline, summarized

1. Picked the dataset and defined the target audience (Data Analyst/BI).
2. Set up a dedicated GitHub account and SSH identity for this project.
3. Scaffolded the Python environment with `uv`.
4. Built and tested the DuckDB ingestion module.
5. Built and tested the aggregation/sampling module.
6. Built and tested the statistics module (chi-square, ANOVA, logistic regression).
7. Scaffolded the documentation vault.
8. Scaffolded the report template.
9. Ran the full pipeline against the real 7.7-million-row dataset.
10. Built and executed the analysis notebook, generating real findings.
11. Built and tested the Streamlit dashboard, later upgraded with a map, year trend, cross-filtering, and a key-drivers panel.
12. Set up GitHub Actions for automated testing (CI).
13. Configured and deployed to Render (CD) — a live, public dashboard.
14. Finalized reports, including this one, as PDFs.

---

## Interview questions and answers

### 1. Why did you choose the US Accidents dataset for this project?

It's large (~7.7 million records) and genuinely well-known, which demonstrates the ability to work with real-world data volume rather than a small toy dataset. It also has a clear business angle — road safety — that's easy for any

interviewer to understand and relate to, which matters for a Data Analyst/BI role where communicating findings to non-technical stakeholders is part of the job.

### 2. Why did you use DuckDB instead of just pandas?

pandas typically needs to load an entire file into memory before you can work with it, which becomes slow or impractical for a multi-gigabyte file. DuckDB is an embedded analytical database that can query a large CSV directly off disk, similar to a full database engine, without needing to run a separate database server. It made the aggregation and sampling queries fast (the full pipeline ran in under a minute) without needing special big-data infrastructure.

### 3. Walk me through your data pipeline architecture.

Raw CSV → DuckDB (for schema validation and fast aggregation/sampling, without loading the whole file into memory) → small Parquet files (pre-aggregated rollups plus a 200,000-row reproducible sample) → those Parquet files feed both the analysis notebook and the dashboard. The raw file itself never leaves the local machine or gets committed to git.

### 4. Why did you take a sample instead of using all 7.7 million rows for your statistical tests?

Statistical hypothesis tests don't need every row to produce a valid, reliable result — a large, well-chosen random sample (200,000 rows here) gives essentially the same statistical power at a fraction of the computational cost. I made the sample reproducible with a fixed random seed, so re-running the analysis always produces the same sample and therefore the same results.

### 5. Explain the difference between a chi-square test and ANOVA — why did you use each here?

A chi-square test checks whether two *categorical* variables are related (I used it for severity category vs. weather category). ANOVA checks whether the *average* of a numeric variable differs across several groups (I used it for average severity across different hours of the day, where hour is the grouping variable). The choice depends on whether you're comparing categories against categories, or a number's average across categories.

### 6. What is a p-value, and what does it mean that your chi-square p-value was reported as "p < 0.0001"?

A p-value estimates the probability of seeing a result at least this extreme *if there were actually no real relationship* in the underlying population. A very small p-value (here, so small it mathematically rounds to zero given the sample size and effect size) means it's extremely unlikely this pattern happened by pure chance — strong evidence of a real relationship between weather and severity. It doesn't tell you *how big or important* the relationship is, only that it's unlikely to be random noise.

### 7. What is an odds ratio, and how do you interpret an odds ratio of 1.34 for "Junction"?

An odds ratio compares the odds of an outcome happening under one condition versus another. An odds ratio of 1.34 for "Junction" means that accidents occurring near a junction have about 34% higher odds of being high-severity compared to accidents not near a junction, holding the other factors in the model constant. An odds ratio below 1 (like 0.41 for "Crossing") means the opposite — lower odds of high severity.

### 8. What is pseudo R-squared, and why was yours so low (0.024) despite statistically significant results?

Pseudo R-squared roughly measures how much of the variation in the outcome the model's inputs explain — lower means the inputs only explain a small share of it. In this project, the model used only four road-feature variables; there are clearly many other factors that affect accident severity (weather, driver behavior, vehicle type, and more) that weren't included in that particular regression. A low pseudo R-squared alongside statistically significant predictors isn't a contradiction — it means the effects are real and reliable, just not the whole story.

### 9. What's the difference between statistical significance and practical/business significance?

Statistical significance (a small p-value) tells you a result is unlikely to be due to random chance in your sample. It does not by itself tell you the effect is *large* or *actionable*. With millions of rows, even tiny, practically unimportant effects can become statistically significant. That's part of why I reported odds ratios and confidence intervals, not just p-values — they show the actual *size* of each effect, which is what a business stakeholder actually needs to make a decision.

### 10. Your findings show associations, not causation — how would you explain that distinction?

Finding that accidents near junctions tend to be more severe doesn't prove junctions *cause* higher severity — there could be other factors (like traffic speed or road design) that explain both. This project uses observational data (not a controlled experiment), so I'm careful to describe findings as associations/predictors, not proven causes, both in the notebook and in the written reports.

### 11. What is Test-Driven Development, and can you give an example of a real bug it caught in this project?

TDD means writing the test for a piece of functionality *before* writing the functionality itself, so you define "correct" precisely before you start. In this project, TDD caught two real bugs immediately: a DuckDB limitation where a certain SQL statement type doesn't support parameterized values (caught by a failing ingestion test), and a subtle bug where a statistics function returned a `numpy.bool_` instead of a plain Python `bool`, caught by a strict identity check in a test.

### 12. Why did you choose Streamlit for the dashboard instead of Tableau or Power BI?

The brief called for a Python project, and Streamlit turns a plain Python script into a fully interactive web app without needing separate BI tooling or licenses — it's a natural fit for a code-first portfolio piece, and it deploys easily to a free hosting service.

### 13. What is CI/CD, and how did you implement it in this project?

CI (Continuous Integration) automatically runs tests and code checks every time code is pushed, so broken code is caught immediately rather than relying on someone remembering to test manually. CD (Continuous Deployment) automatically publishes the latest working code. I implemented CI with a GitHub Actions workflow that runs the test suite and linter on every push, and CD by connecting the GitHub repo to Render, which automatically rebuilds and republishes the live dashboard whenever new code lands on the main branch.

### 14. Why did you decide against using Docker for deployment?

Docker is valuable when an app has a complex environment to reproduce exactly, but this dashboard's only real dependency at runtime is reading small pre-built data files with pandas and displaying charts — there was nothing that needed containerizing. Render can run the Python app directly with a simple build and start command, which is simpler to set up and maintain without a meaningful loss of reliability.

### 15. How did you ensure the raw 3GB dataset never got committed to git, and why does that matter?

I added the raw data folder to `.gitignore` from the very start of the project, before any data existed locally, so it was never possible to accidentally commit it. This matters because GitHub blocks files over 100MB outright, and even if it didn't, committing multi-gigabyte files would make the repository slow to clone and review — bad practice regardless of file size limits.

### 16. What tradeoffs did you make given the time constraint of "a few days"?

I scoped the regression model to four clearly interpretable road-feature predictors rather than a larger, more complex model; used pre-aggregated rollups and a reproducible sample instead of processing the full dataset for every operation; and chose Render's simplest deployment path (no Docker) to minimize setup time without sacrificing the ability to demonstrate real CI/CD.

### 17. If you had more time, what would you add or improve?

I'd add more predictors to the regression model (weather, time of day, geography) to build a more complete picture of what drives severity; add a proper time-series analysis of the year-over-year trend; and consider adding user authentication or usage analytics to the dashboard if it were being used by a real team rather than as a portfolio piece.

### 18. How would you explain your key finding to a non-technical stakeholder?

"Where an accident happens matters. Accidents near junctions tend to be worse, while accidents near crossings, traffic signals, or stop signs tend to be less severe — likely because those features already slow traffic down. This suggests that road design and signage placement are worth investigating further as a lever for reducing accident severity."

### 19. What was the most challenging technical problem you ran into, and how did you solve it?

Slow package installs, traced back to the project living on an external drive formatted as exFAT, which doesn't support the filesystem feature (hardlinking) that normally makes Python environment installs nearly instant. I diagnosed it by reading the actual tool warning message, checked the drive's filesystem type directly to confirm the real cause, and then made a deliberate, informed decision to accept the slower installs rather than restructure the project — a good example of not just applying a fix blindly, but understanding the actual root cause before deciding whether it's worth fixing at all.

### 20. How do you keep your analysis reproducible?

Every randomized step (the 200,000-row sample) uses a fixed random seed. Every interpretive sentence in the notebook is generated directly from the computed statistics, not typed by hand, so it can never drift out of sync with the actual numbers. And the entire pipeline — from raw data to final dashboard — is captured in version-controlled code with automated tests, so anyone can re-run it and get the same result.